

SOFTWARE REQUIREMENTS SPECIFICATION

Game-Based Learning Platform

for Teaching Sorting Algorithms

Using Interactive 3D Cubes

Sana Fatima (245207)

Hajraa Noor (245225)

Spring 2026 · Software Engineering · Air University AACK Kamra

University Name:	Air University Aerospace & Aviation Campus Kamra
Department:	Department of Computer Science
Course:	Software Engineering
Document Type:	Software Requirements Specification
Submitted By:	Sana Fatima (245207), Hajraa Noor (245225)
Submitted To:	Mam Sahar Maqsood Ul Hassan
Semester:	Spring 2026
Date:	20th May 2026

Approval Page

Name	Designation	Signature
Mam Sahar Maqsood Ul Hassan	Project Supervisor	_____
Sir Rehman Gul	Course Coordinator	_____

Dedication

This work is dedicated to my parents and teachers, whose patience and encouragement made this journey possible. It is also dedicated to every student who has ever sat in a classroom and struggled to understand an algorithm from a whiteboard alone. This platform was built for you.

Acknowledgement

I would like to begin by expressing my deepest gratitude to Allah Almighty, without Whose blessings and guidance no effort bears fruit. I am sincerely grateful to my project supervisor, Mam Sahar Maqsood Ul Hassan, for her continuous guidance, timely feedback, and encouragement throughout the development of this project. Their academic expertise and dedication have been invaluable in shaping both the technical depth and the professional presentation of this work.

I also extend my gratitude to the faculty members of the Department of Computer Science for their constructive advice during progress reviews. My heartfelt thanks go to my family for their constant moral support, and to my fellow students who served as early testers of the platform. This project would not have reached completion without the collaborative spirit of everyone involved.

Abstract

The teaching of fundamental computer science algorithms has long relied on passive methods such as whiteboard illustrations and static textbook diagrams. This approach frequently results in students who can recite an algorithm's steps without truly understanding the logic behind each operation.

The **Game-Based Learning Platform for Teaching Sorting Algorithms** is a Unity-based interactive educational game developed in C# that directly addresses this problem. The platform presents sorting algorithms — specifically Bubble Sort, Selection Sort, and Insertion Sort — as interactive puzzle challenges where the student becomes the algorithm itself.

The system generates a randomised array represented as interactive three-dimensional numbered cubes. The player must manually perform each sorting step by selecting and swapping cubes. Every move the player makes is validated in real time against the actual rules of the selected algorithm by the internal **SortingManager**. The platform further supports structured learning through a dedicated Learning Panel that presents theory, pseudocode, and animated examples before gameplay begins.

The application is built on the Unity Engine, coded in C#, and follows a Singleton-based Manager architecture with fully separated layers for UI, game logic, algorithm computation, and data persistence.

Contents

Approval Page	1
Dedication	2
Acknowledgement	3
Abstract	4
Abbreviations and Acronyms	9
Change Log / Revision History	10
1 Introduction	11
1.1 Purpose of Document	11
1.2 Project Scope	11
1.3 Problem Statement	12
1.4 Objectives	12
1.5 Proposed Solution	13
1.6 Intended Users	13
1.7 Development Tools and Technologies	14
1.8 Assumptions and Dependencies	14
1.9 Constraints	14
2 Literature Review	16
2.1 Introduction	16
2.2 Existing Systems	16
2.2.1 VisuAlgo	16
2.2.2 HackerRank	16
2.2.3 CS Unplugged / Algorithm Treasure Hunt (Research Literature)	17
2.3 Comparative Analysis	18
2.4 Research Gap	18
2.5 Summary	18
3 System Modelling and Design	19
3.1 System Overview	19
3.2 Functional Requirements	19
3.2.1 FR-01: Algorithm Selection	19
3.2.2 FR-02: Learning Material Display	20
3.2.3 FR-03: Level Generation	20
3.2.4 FR-04: Move Validation Engine	21
3.2.5 FR-05: Timer and Scoring System	21
3.2.6 FR-06: Guided Practice with Hints	22
3.2.7 FR-07: Challenge Mode	22
3.2.8 FR-08: Result and Statistics Screen	23
3.2.9 FR-09: Audio Feedback System	23
3.2.10 FR-10: Data Persistence	24
3.3 Non-Functional Requirements	24
3.3.1 Performance Requirements	24
3.3.2 Security Requirements	24
3.3.3 Reliability Requirements	25
3.3.4 Usability Requirements	25

3.3.5	Scalability Requirements	25
3.4	Use Case Diagram and Specifications	25
3.4.1	Use Case Description	25
3.4.2	Use Case Specifications	27
3.5	Activity Diagrams	29
3.5.1	Core Gameplay Loop Activity Diagram	29
3.5.2	Algorithm Selection and Learning Flow Activity Diagram	30
3.6	Sequence Diagrams	31
3.6.1	Move Validation Sequence Diagram	31
3.6.2	Level Initialisation Sequence Diagram	32
3.7	Class Diagram	33
3.8	Domain Model	34
3.9	Entity Relationship Diagram	35
3.9.1	Database Design	35
3.9.2	Database Tables	36
3.10	Package Diagram	37
3.11	Architecture Diagram	37
3.12	Deployment Diagram	38
4	Testing	39
4.1	Testing Introduction	39
4.2	Testing Strategy	39
4.3	Test Environment	39
4.4	Test Cases	40
4.5	Functional Testing	42
4.6	Integration Testing	42
4.7	System Testing	43
4.8	Test Results	43
4.9	Limitations of Testing	43
5	Conclusion and Future Work	44
5.1	Conclusion	44
5.2	Future Enhancements	44
A	Glossary of Key Terms	46
B	Source Code Snippet: Move Validation	47
C	User Manual	48
C.1	Installation (Desktop)	48
C.2	Installation (Android)	48
C.3	How to Play	48

List of Figures

1	use-case diagram	26
2	Activity Diagram — Core Gameplay Loop	30
3	Activity Diagram — Algorithm Selection and Learning Flow	31
4	Sequence Diagram — Move Validation Process	32
5	Sequence Diagram — Level Initialisation.....	33
6	Class Diagram — Core Manager Architecture	34
7	Domain Model	35
8	Entity Relationship Diagram — Local Storage.....	36
9	Package Diagram	37
10	System Architecture Diagram	38
11	Deployment Diagram	38

List of Tables

1	Intended Users	13
2	Development Tools and Technologies	14
3	Feature Comparison: VisuAlgo	16
4	Feature Comparison: HackerRank / LeetCode	17
5	Feature Comparison: CS Unplugged	17
6	Comparative Analysis: All Systems	18
7	FR-01: Algorithm Selection	19
8	FR-02: Learning Material Display	20
9	FR-03: Level Generation	20
10	FR-04: Move Validation Engine	21
11	FR-05: Timer and Scoring System	21
12	FR-06: Guided Practice with Hints	22
13	FR-07: Challenge Mode	22
14	FR-08: Result and Statistics Screen	23
15	FR-09: Audio Feedback System	23
16	FR-10: Data Persistence	24
17	NFR: Performance Requirements	24
18	NFR: Security Requirements	24
19	NFR: Reliability Requirements	25
20	NFR: Usability Requirements	25
21	NFR: Scalability Requirements	25
22	UC-01: Select Algorithm	27
23	UC-02: Study Learning Material	27
24	UC-03: Play Guided Practice Level	28
25	UC-04: Select and Swap Cubes	28
26	UC-05: Receive Move Feedback	29
27	USER_PROFILE Table	36
28	ALGORITHM_STATS Table	37
29	Test Environment	39
30	TC-01: Bubble Sort Valid Move	40
31	TC-02: Bubble Sort Invalid Move	40
32	TC-03: Selection Sort Move Validation	40
33	TC-04: Timer Timeout Handling	41
34	TC-05: Level Completion Trigger	41
35	TC-06: Score and Accuracy Calculation	41
36	TC-07: Data Persistence Across Sessions	42
37	TC-08: Guided Practice Hint Display	42

Abbreviations and Acronyms

Term	Meaning
SRS	Software Requirements Specification
UML	Unified Modelling Language
UI	User Interface
HUD	Heads-Up Display
OOP	Object-Oriented Programming
ERD	Entity Relationship Diagram
FPS	Frames Per Second
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
TC	Test Case
C#	C Sharp (Programming Language)

Change Log / Revision History

Version	Date	Description	Author
1.0	18-05-2026	Initial draft - complete SRS with all chapters	Sana Fatima
1.1	19-05-2026	Diagrams and testing chapter finalised	Hajraa Noor
1.2	26-05-2026	Enhanced document formatting: applied professional colour scheme (navy blue, royal blue, gold accents), styled cover page, colour-coded tables and headings, added full Table of Contents, List of Figures, and List of Tables	Sana Fatima

Introduction

This chapter establishes the context, purpose, and scope of the Game-Based Learning Platform for Sorting Algorithms. It begins with the motivation behind the project, outlines the problem it seeks to solve, and presents the proposed solution along with the technical objectives that guide its development.

Purpose of Document

This Software Requirements Specification (SRS) serves as the definitive technical document governing the design, development, and evaluation of the Game-Based Learning Platform. Its primary purpose is to communicate — in formal, unambiguous language — every functional expectation, non-functional quality standard, and architectural decision associated with the system.

The document is intended for multiple audiences. Project supervisors and evaluators will use it to assess whether the final product faithfully realises the stated requirements. Developers will reference it during the implementation phase to ensure consistency between design intent and the built system. Testers will draw from its functional requirements and test case tables to structure their validation activities. Students and teachers who use the final product may also consult the earlier chapters to understand what the platform is designed to achieve.

Project Scope

The Game-Based Learning Platform is a standalone educational game application. It operates independently, requiring no network connectivity or server infrastructure for its core functionality. The system is targeted primarily at undergraduate computer science students learning algorithms for the first time, though its accessible design also makes it suitable for self-learners and high school students with an interest in programming.

The platform encompasses the following major functional modules:

- A **Learning Module** that displays algorithm theory, pseudocode, complexity analysis (Big-O notation), and an animated demonstration for each supported sorting algorithm.
- A **Challenge Mode Module** where the player must independently recall and execute the correct algorithm steps without any hint assistance.
- An **Evaluation and Results Module** that presents the player's score, accuracy percentage, time taken, and a performance grade upon completion.
- A **Data Persistence Module** that records high scores and per-algorithm statistics between sessions using local storage.

The system currently supports three algorithms — Bubble Sort, Selection Sort, and Insertion Sort — with the architecture deliberately designed to allow future algorithms to be added with minimal refactoring.

Problem Statement

In most computer science curricula, sorting algorithms are introduced using static diagrams drawn on a whiteboard or printed in textbooks. A typical lecture might show a before-and-after snapshot of an array transformation or trace through a few steps manually. While this approach conveys the idea of an algorithm, it does little to help students internalise the *decision-making logic* behind each step.

The core difficulty is cognitive: students must simultaneously track the state of a changing array, remember the rule governing the current algorithm, and determine which element should be moved next. Without active practice, students often confuse algorithms, forget their rules under examination conditions, or fail to grasp why certain algorithms are more efficient than others.

Existing visualisation tools, such as VisuAlgo, present algorithm animations but do not require the student to make any decisions. The student watches the algorithm run rather than running it themselves. Platforms like LeetCode, on the other hand, require the student to write code — a significant barrier for learners who have not yet developed programming fluency. There is a clear and unaddressed need for an interactive middle ground: a system that requires active, step-by-step participation in algorithm execution, without the prerequisite of coding skill.

Objectives

The development of this platform is guided by the following specific objectives:

- To design and implement an interactive educational game using the Unity Engine and C# that visually represents sorting algorithms as a manipulable cube puzzle.
- To implement a real-time move validation engine that compares each player action against the pre-computed correct steps of the selected algorithm, ensuring educational accuracy.
- To improve student engagement and motivation through gamification mechanics including a scoring system, countdown timer, mistake tracking, and performance grades.
- To structure the learning experience progressively — from passive theory consumption, to guided interactive practice, to unassisted challenge mode — in order to build genuine algorithmic understanding.
- To reduce the conceptual difficulty of sorting algorithms for first-year computer science students by providing immediate, visual, and auditory feedback on every action.
- To build a scalable software architecture that allows new sorting algorithms to be added in future iterations without redesigning the core gameplay or user interface.

Proposed Solution

The proposed solution is a Unity-based educational game in which sorting algorithms are taught through hands-on cube manipulation. Upon launching the application, the user navigates to an algorithm selection screen, where they choose one of the supported algorithms. The system then presents a Learning Panel containing a clear written explanation of the algorithm, its Big-O time complexity for best, average, and worst cases, a pseudocode representation, and an animated visual example.

After studying the material, the player progresses to the Actual Game Panel. A set of three-dimensional numbered cubes — representing an unsorted integer array — is generated with random values. The internal `SortingManager` script, which has pre-computed the complete correct move sequence before the level began, immediately validates whether the player's swap matches the expected algorithm step. A correct move triggers a positive visual and audio response and increments the score. An incorrect move triggers a negative response and increments the mistake counter.

Upon sorting the array completely, the Result Screen appears, displaying a comprehensive performance summary and a letter grade based on accuracy.

Intended Users

Table 1: Intended Users

User Type	Description
Student / Player	The primary user who engages with the learning content and gameplay to study sorting algorithms.
Educator / Teacher	A secondary user who may use the platform as a classroom demonstration tool to make algorithm teaching more visual and interactive.

Development Tools and Technologies

Table 2: Development Tools and Technologies

Category	Technology	Purpose
Game Engine	Unity Engine (2022 LTS)	Core platform for game development, rendering, and scene management
Programming Language	C# (C Sharp)	All game logic, validation, UI control, and data management
Animation Library	DOTween (DG.Tweening)	Smooth cube swap animations and UI transitions
UI Framework	TextMeshPro (Unity UI)	All in-game text rendering including scores, timers, and labels
IDE	Visual Studio	Code editing and debugging
Diagram Tool	Draw.io	Producing all UML and architectural diagrams
Version Control	Unity Version Control	Source code management and team collaboration

Assumptions and Dependencies

- The player's device must meet the minimum hardware specification to run a Unity standalone application, including a GPU capable of basic 3D rendering.
- The DOTween animation library must be correctly imported into the Unity project for all visual transitions to function as described.
- The system is designed to operate without internet connectivity. The local **SaveSystem** module handles all data persistence using Unity's PlayerPrefs or local JSON file storage.
- It is assumed that users possess basic device interaction skills, such as the ability to click or tap on interface elements.

Constraints

- **Time Constraint:** The project must be completed within the academic semester timeline, limiting the number of algorithms and features that could be implemented in version 1.0.
- **Hardware Constraint:** The application is designed for standard Android hardware. It has not been optimised for very low-end devices or unusual screen resolutions.
- **Scope Constraint:** The initial release supports three sorting algorithms. Advanced algorithms such as Merge Sort and Quick Sort are deferred to future development

phases, as their divide-and-conquer logic requires significant extensions to the cube interaction model.

Literature Review

Introduction

This chapter examines existing tools and research that relate to the teaching of algorithms through digital and interactive means. The review focuses on platforms that attempt to make algorithm education more accessible, whether through visualisation, gamification, or code practice. The analysis of their capabilities and limitations provides the intellectual foundation for the design choices made in this project.

Existing Systems

VisuAlgo

VisuAlgo is a web-based algorithm visualisation platform developed at the National University of Singapore. It provides animated, step-by-step walkthroughs of over forty algorithms, including all standard sorting algorithms, graph algorithms, and data structure operations. The animations are accompanied by pseudocode highlighting that shows which line of code corresponds to each visual step.

Table 3: Feature Comparison: VisuAlgo

Feature	Available
Step-by-step algorithm animation	Yes
Pseudocode with line highlighting	Yes
Active user participation	No
Gamification (score, timer)	No
Performance evaluation	No
Works without coding knowledge	Yes

Advantages: Comprehensive coverage of algorithms. Clean and academic visual style. Freely accessible with no installation required.

Limitations: Entirely passive. The student observes the algorithm executing but never makes a decision themselves. There is no mechanism to verify whether a student has understood the material or can replicate the algorithm independently.

HackerRank

HackerRank is a competitive programming platform where users solve algorithm challenges by writing code that must pass automated test cases. It provides a rich library of problems organised by topic and difficulty.

Table 4: Feature Comparison: HackerRank / LeetCode

Feature	Available
Algorithm visualisation	No
Active user participation	Yes (via code)
Gamification (badges, ranks)	Yes
Performance evaluation	Yes
Accessible to non-programmers	No

Advantages: Develops genuine implementation skills. Provides automated feedback through test cases. Supports a wide range of difficulty levels.

Limitations: Requires programming proficiency as a prerequisite. A student cannot use HackerRank to learn an algorithm at a conceptual level; they must already understand both the algorithm and a programming language well enough to implement it correctly.

CS Unplugged / Algorithm Treasure Hunt (Research Literature)

A body of educational research supports the concept of “unplugged” algorithm learning, where students physically perform algorithm steps using cards or blocks in a classroom setting. Studies by Bell, Witten, and Fellows (1998) demonstrated that physical enactment of algorithms significantly improves retention compared to observation alone. However, these activities require a physical classroom, a teacher facilitator, and materials preparation, making them impractical for self-directed or remote learning.

Table 5: Feature Comparison: CS Unplugged

Feature	Available
Algorithm visualisation	Limited
Active user participation	Yes (physical)
Scalable to self-study	No
Instant feedback	No
Gamification	Minimal

Advantages: Proven pedagogical effectiveness. Forces active decision-making about algorithm steps.

Limitations: Not a software tool. Requires a physical space and human facilitation. Cannot scale to remote or asynchronous learning contexts.

Comparative Analysis

Table 6: Comparative Analysis: All Systems

Feature	VisuAlgo	HackerRank	CS Unplugged	Proposed
Algorithm Visualisation	High	None	Medium	High
Active Student Participation	None	High	High	High
Prerequisite: Coding Knowledge	No	Required	Not req.	Not req.
Real-time Feedback	None	Yes	Limited	Yes
Gamification	None	Partial	None	Full
Scalable to Self-Study	Yes	Yes	No	Yes
Educational Accuracy Enforced	No	Yes	Yes	Yes
Mobile / Standalone App	No	No	No	Yes

Research Gap

The review of existing systems reveals a significant and consistent gap in the educational technology landscape. Passive visualisation tools are educationally insufficient because they do not require the student to demonstrate understanding through action. Code-based platforms are educationally inaccessible for students still in the early stages of learning algorithms. Physical classroom activities, while pedagogically sound, cannot scale beyond a traditional learning environment.

The proposed platform fills this gap by combining the visual clarity of tools like VisuAlgo with the active, decision-demanding participation model validated by research in physical algorithm enactment, delivered in a scalable, self-contained digital application. Critically, the real-time move validation engine means that unlike any of the reviewed systems, the platform can detect and correct a student's misunderstanding of an algorithm at the exact moment it occurs.

Summary

Existing solutions for algorithm education represent two extremes: passive observation on one end, and active code implementation on the other. Neither extreme serves the needs of students who are encountering algorithms for the first time and need a structured, interactive space to practise decision-making at an algorithmic level. This review confirms that the proposed Game-Based Learning Platform occupies a unique and educationally valuable position in this space.

System Modelling and Design

System Overview

The Game-Based Learning Platform follows a Singleton-based Manager architecture, a well-established Unity design pattern in which a small number of centralised manager objects control the major subsystems of the application. The frontend Unity scenes handle user input and visual rendering. User interactions are routed to the appropriate manager. The managers delegate computational tasks — particularly algorithm step validation — to specialised logic classes, and persist results to the local data layer.

The system is organised into six Unity scenes: a Main Menu, an Algorithm Selection Screen, a Learning Panel and a Gameplay Scene. A persistent `GlobalManager` object, configured with `DontDestroyOnLoad()`, maintains the audio system, background video, and globally shared state across all scene transitions.

Functional Requirements

The following tables define the functional requirements of the system. Each requirement is assigned a unique identifier, a priority level (Critical, High, Medium, or Low), and a precise description of the expected system behaviour.

FR-01: Algorithm Selection

Table 7: FR-01: Algorithm Selection

Field	Description
Requirement ID	FR-01
Requirement Name	Algorithm Selection
Description	The system must present the user with a selection of available sorting algorithms. Upon selection, the chosen algorithm must be stored globally and used to configure the Learning Panel and the Gameplay validation engine.
Input	Mouse click or screen tap on an algorithm card (Bubble Sort, Selection Sort, Insertion Sort).
Output	Global algorithm state is updated. The user is navigated to the Learning Panel configured for the selected algorithm.
Priority	Critical

FR-02: Learning Material Display*Table 8: FR-02: Learning Material Display*

Field	Description
Requirement ID	FR-02
Requirement Name	Learning Material Display
Description	The Learning Panel must display: (a) a plain-language description of how the algorithm works, (b) the best-case, average-case, and worst-case time complexity in Big-O notation, (c) a pseudocode representation, and (d) an animated visual demonstration showing at least one complete sorting pass.
Input	Selected algorithm state from FR-01.
Output	TextMeshPro UI components update to display algorithm-specific content. Animation coroutine begins automatically upon panel load.
Priority	High

FR-03: Level Generation*Table 9: FR-03: Level Generation*

Field	Description
Requirement ID	FR-03
Requirement Name	Level Generation
Description	Upon loading the Gameplay Scene, the system must generate a randomised array of integers and instantiate a corresponding set of 3D cube prefabs. Each cube must visually display its assigned integer value. The number of cubes must correspond to the selected number. The generated array must not already be sorted.
Input	Scene load event, selected difficulty level.
Output	An array of randomised integers is stored internally. Cube GameObjects are instantiated in a horizontal row at pre-defined positions.
Priority	Critical

FR-04: Move Validation Engine*Table 10: FR-04: Move Validation Engine*

Field	Description
Requirement ID	FR-04
Requirement Name	Move Validation Engine
Description	This is the most critical functional requirement of the system. Before gameplay begins, the SortingManager must pre-compute the complete ordered sequence of valid swap operations for the current array using the selected algorithm. When the player selects two cubes, GameManager must call SortingManager.ValidateMove(indexA, indexB) . If the player's selected indices match the next expected move, the move is accepted, cubes are swapped visually, and the score is incremented. If the move does not match, it is rejected and the mistake counter is incremented.
Input	Two sequential cube selections by the player (indexA, indexB).
Output	Boolean result (valid or invalid). Visual feedback (green glow / red flash), audio feedback, score or mistake counter update.
Priority	Critical

FR-05: Timer and Scoring System*Table 11: FR-05: Timer and Scoring System*

Field	Description
Requirement ID	FR-05
Requirement Name	Timer and Scoring System
Description	A countdown timer must begin when the gameplay level loads. The initial time value is 120 seconds. Initial Score is +20. If the timer reaches zero before the array is sorted, the level must end in a timeout state. Each validated correct move awards +10 points. Each rejected incorrect move deducts 5 points.
Input	Time elapsed, validated/rejected move events.
Output	Real-time HUD update of timer, score, and mistake count. Level-end event trigger on timeout.
Priority	High

FR-06: Guided Practice with Hints*Table 12: FR-06: Guided Practice with Hints*

Field	Description
Requirement ID	FR-06
Requirement Name	Guided Practice with Hints
Description	In the Guided Practice Panel, the system must display a contextual hint describing the next required action. For Bubble Sort: indicate which adjacent pair to compare. For Selection Sort: indicate the current minimum element. For Insertion Sort: indicate where the current element should be inserted. The relevant cubes must also be visually highlighted (outline glow).
Input	Current algorithm state, <code>currentMoveIndex</code> value from <code>SortingManager</code> .
Output	Hint text updated in UI. Cube highlight effect activated on the relevant cubes.
Priority	High

FR-07: Challenge Mode*Table 13: FR-07: Challenge Mode*

Field	Description
Requirement ID	FR-07
Requirement Name	Challenge Mode
Description	The Challenge Mode Panel must use the same core gameplay mechanics as Guided Practice but must not display any hint text or cube highlighting. The player must rely entirely on their memory of the algorithm to determine correct moves. The move validation engine (FR-04) remains active and enforces the same accuracy requirements.
Input	Player interaction events.
Output	Same as FR-04 and FR-05, with hint and highlight systems disabled.
Priority	High

FR-08: Result and Statistics Screen*Table 14: FR-08: Result and Statistics Screen*

Field	Description
Requirement ID	FR-08
Requirement Name	Result and Statistics Screen
Description	Upon level completion, the system must display: (a) the algorithm used, (b) the final score, (c) total correct moves, (d) total incorrect moves. Two buttons must be present: Restart and Back.
Input	Completion event from <code>GameManager</code> , final statistics from <code>ScoreManager</code> .
Output	Result Scene populated with formatted statistics and navigation buttons active.
Priority	High

FR-09: Audio Feedback System*Table 15: FR-09: Audio Feedback System*

Field	Description
Requirement ID	FR-09
Requirement Name	Audio Feedback System
Description	The system must play distinct audio clips in response to user actions and game events: background music (persistent across scenes), a button click sound, a correct-move success sound, an incorrect-move error sound, and a level-completion fanfare. The player must be able to mute or adjust volume from the settings menu.
Input	Gameplay events (correct move, incorrect move, level complete, button press).
Output	Corresponding audio clip plays via <code>AudioManager.PlaySound()</code> .
Priority	Medium

*FR-10: Data Persistence**Table 16: FR-10: Data Persistence*

Field	Description
Requirement ID	FR-10
Requirement Name	Data Persistence
Description	The SaveSystem module must persist between sessions: (a) the player's all-time high score, (b) the number of completions per algorithm, and (c) the highest accuracy achieved per algorithm. Data must be stored using Unity's PlayerPrefs or a JSON file on the local device. Data must be loaded when the Main Menu initialises and saved immediately after each level completion.
Input	Level completion events with final statistics.
Output	Persistent data written to local storage. Statistics Screen and Main Menu load and display historical best values.
Priority	Medium

Non-Functional Requirements

*Performance Requirements**Table 17: NFR: Performance Requirements*

ID	Description
NFR-01	The application must maintain a stable frame rate of no less than 60 FPS on any device that meets the minimum hardware specification, ensuring animations and interactions remain fluid at all times.
NFR-02	The cube swap animation, powered by DOTween, must complete within 0.35 seconds. This duration is short enough to maintain gameplay momentum while remaining visually comprehensible.
NFR-03	Scene load times must not exceed 3 seconds on target hardware. Excessive loading times break the player's engagement and detract from the learning experience.

*Security Requirements**Table 18: NFR: Security Requirements*

ID	Description
NFR-04	Local save data stored via the SaveSystem must be written atomically to prevent partial writes that could corrupt the player's progress data in the event of an unexpected application closure.
NFR-05	The application must not request or access any network resources, personal data, or device permissions beyond local file system access for save data.

Reliability Requirements

Table 19: NFR: Reliability Requirements

ID	Description
NFR-06	The <code>SortingManager</code> 's pre-computed move list must be 100% algorithmically accurate. Any error in the validation logic would directly mislead students and undermine the educational purpose of the platform.
NFR-07	The application must handle edge cases in level generation gracefully. If a random array is generated in already-sorted order, the system must detect this and regenerate the array before displaying it to the player.

Usability Requirements

Table 20: NFR: Usability Requirements

ID	Description
NFR-08	The UI must be intuitive for a first-time user without any tutorial beyond the in-game Learning Panel. Font sizes, button sizes, and colour contrasts must meet basic accessibility standards.
NFR-09	Visual feedback on cube selection must be immediate — within one frame of the tap or click — so the player always has clear confirmation of their current selection state.
NFR-10	All text displayed in the Learning Panel must be written at a readability level appropriate for first-year university students, avoiding unnecessary jargon.

Scalability Requirements

Table 21: NFR: Scalability Requirements

ID	Description
NFR-11	The algorithm system must be architected so that a new sorting algorithm can be added by creating a single new C# class that inherits from a base sorting logic class and implements a <code>GenerateMoves()</code> method, without any changes to <code>GameManager</code> , <code>CubeController</code> , or <code>ScoreManager</code> .
NFR-12	Difficulty levels must be configurable through a data-driven approach (e.g., a <code>ScriptableObject</code> or configuration file) rather than hard-coded values, allowing new difficulty tiers to be added without code changes.

Use Case Diagram and Specifications

Use Case Description

The Game-Based Learning Platform has one primary actor: the **Player**. The player interacts with the system through a defined set of use cases that cover the complete learning journey from algorithm selection through to results review.

Actor: Player

Use Cases:

- UC-01: Select Algorithm
- UC-02: Study Learning Material
- UC-03: Play Guided Practice Level (includes: UC-04, UC-05)
- UC-04: Select and Swap Cubes
- UC-05: Receive Move Feedback
- UC-06: Play Challenge Mode Level (includes: UC-04, UC-05)
- UC-07: View Result and Statistics

Relationships:

- UC-03 and UC-06 both include UC-04 (cube selection) and UC-05 (feedback).
- UC-03 extends to UC-06 (Guided Practice is a prerequisite to Challenge Mode in the normal flow).

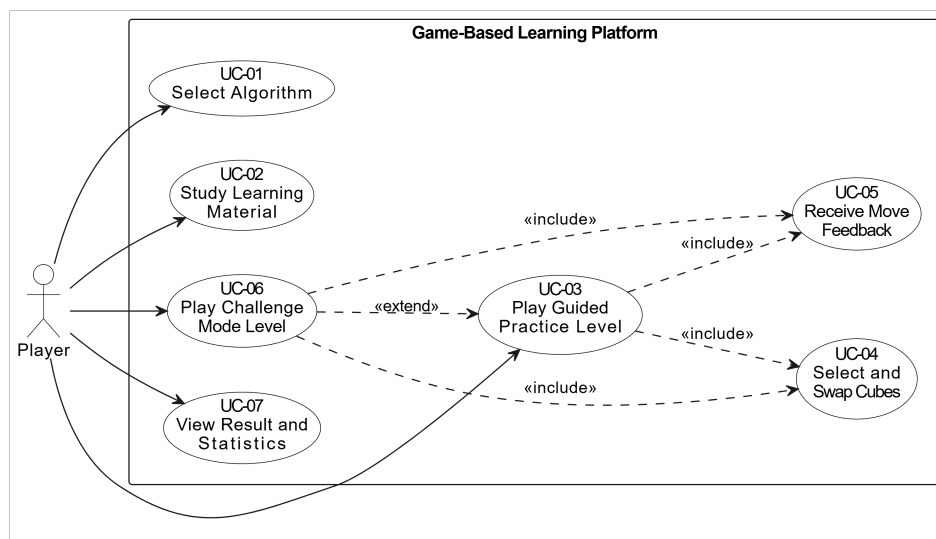


Figure 1: use-case diagram

*Use Case Specifications**Table 22: UC-01: Select Algorithm*

Field	Description
Use Case ID	UC-01
Use Case Name	Select Algorithm
Primary Actor	Player
Preconditions	The player is on the Algorithm Selection Screen. At least one algorithm is available.
Postconditions	The selected algorithm is stored in global state. The Learning Panel loads with the appropriate content.
Basic Flow	1. Player views the available algorithm cards. 2. Player taps/clicks one card. 3. System highlights the selected card. 4. System loads the Learning Panel.
Alternative Flow	Player presses the Back button and returns to the Main Menu without selecting.
Exception Flow	None applicable for this use case.

Table 23: UC-02: Study Learning Material

Field	Description
Use Case ID	UC-02
Use Case Name	Study Learning Material
Primary Actor	Player
Preconditions	An algorithm has been selected (UC-01 completed). Learning Panel is loaded.
Postconditions	Player has reviewed the theory, pseudocode, and animation. Player taps Next to proceed.
Basic Flow	1. Player reads the algorithm description. 2. Player views the animated example. 3. Player reads the complexity table. 4. Player taps “Start Practice”. 5. System loads the Guided Practice Panel.
Alternative Flow	Player taps “Back” to return to Algorithm Selection.
Exception Flow	None.

Table 24: UC-03: Play Guided Practice Level

Field	Description
Use Case ID	UC-03
Use Case Name	Play Guided Practice Level
Primary Actor	Player
Preconditions	Learning Panel has been completed. Gameplay Scene is loaded.
Postconditions	Array is sorted OR timer has expired. Statistics are recorded. Result Scene is loaded.
Basic Flow	1. System spawns randomised cubes and pre-computes valid moves. 2. Hint text and highlighted cubes indicate the next step. 3. Player selects two cubes (UC-04). 4. System validates the move and provides feedback (UC-05). 5. Steps 3–4 repeat until array is sorted or timer expires. 6. Result Scene loads.
Alternative Flow	Player presses Pause and taps Quit to return to Main Menu.
Exception Flow	If timer reaches 0, level ends with timeout status regardless of sort progress.

Table 25: UC-04: Select and Swap Cubes

Field	Description
Use Case ID	UC-04
Use Case Name	Select and Swap Cubes
Primary Actor	Player
Preconditions	Gameplay Scene is active and array is not yet sorted.
Postconditions	Two cubes have been selected. <code>GameManager.AttemptSwap()</code> has been called.
Basic Flow	1. Player clicks/taps Cube A. 2. Cube A glows to indicate selection. 3. Player clicks/taps Cube B. 4. <code>GameManager</code> receives both selections and calls <code>AttemptSwap()</code> .
Alternative Flow	Player clicks the same cube twice; second click deselects and resets selection state.
Exception Flow	Player clicks a non-cube UI element; selection state resets.

Table 26: UC-05: Receive Move Feedback

Field	Description
Use Case ID	UC-05
Use Case Name	Receive Move Feedback
Primary Actor	Player
Preconditions	<code>AttemptSwap()</code> has been called (UC-04 completed).
Postconditions	Player has received visual and auditory feedback. Score or mistake counter has been updated.
Basic Flow	1. <code>GameManager</code> calls <code>SortingManager.ValidateMove()</code> . 2. If valid: cubes animate to swapped positions, green effect plays, success sound plays, score increments. 3. If invalid: red flash effect plays, error sound plays, mistake counter increments. 4. HUD values update. 5. System checks if array is sorted; if yes, level ends.
Alternative Flow	None.
Exception Flow	None.

Activity Diagrams

Core Gameplay Loop Activity Diagram

Figure 2 illustrates the complete activity flow within the Gameplay Scene, from level initialisation through to the end condition. The diagram begins with the system loading the scene, generating random numbers, spawning cube objects, and pre-computing the algorithm's move sequence. When two cubes are selected, the validation check branches the flow: a valid move triggers a swap animation and a completion check, while an invalid move triggers an error response.

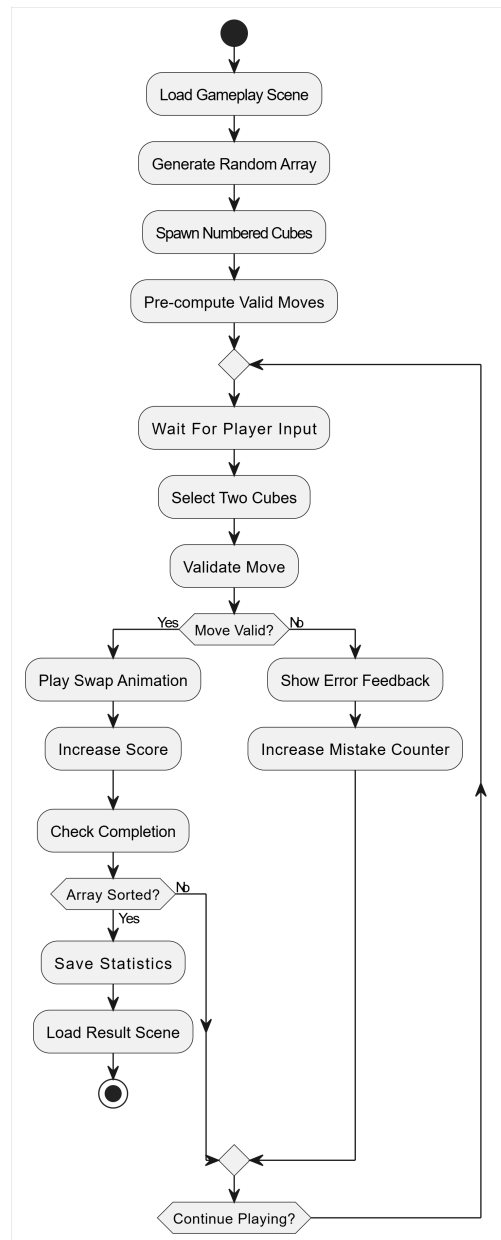


Figure 2: Activity Diagram — Core Gameplay Loop

Algorithm Selection and Learning Flow Activity Diagram

Figure 3 illustrates the flow from the Main Menu through to the start of gameplay, covering algorithm selection and the learning preparation phase.

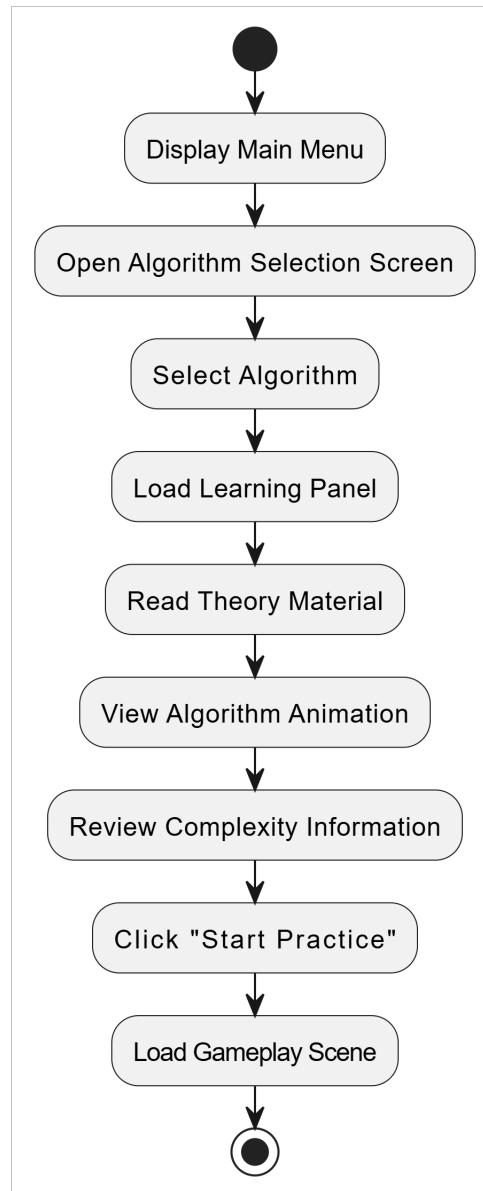


Figure 3: Activity Diagram — Algorithm Selection and Learning Flow

Sequence Diagrams

Move Validation Sequence Diagram

Figure 4 illustrates the precise sequence of message exchanges between system objects when a player attempts a cube swap, from the first cube tap to the final HUD update. The *alt* frame shows two conditional branches: a valid move route and an invalid move route.

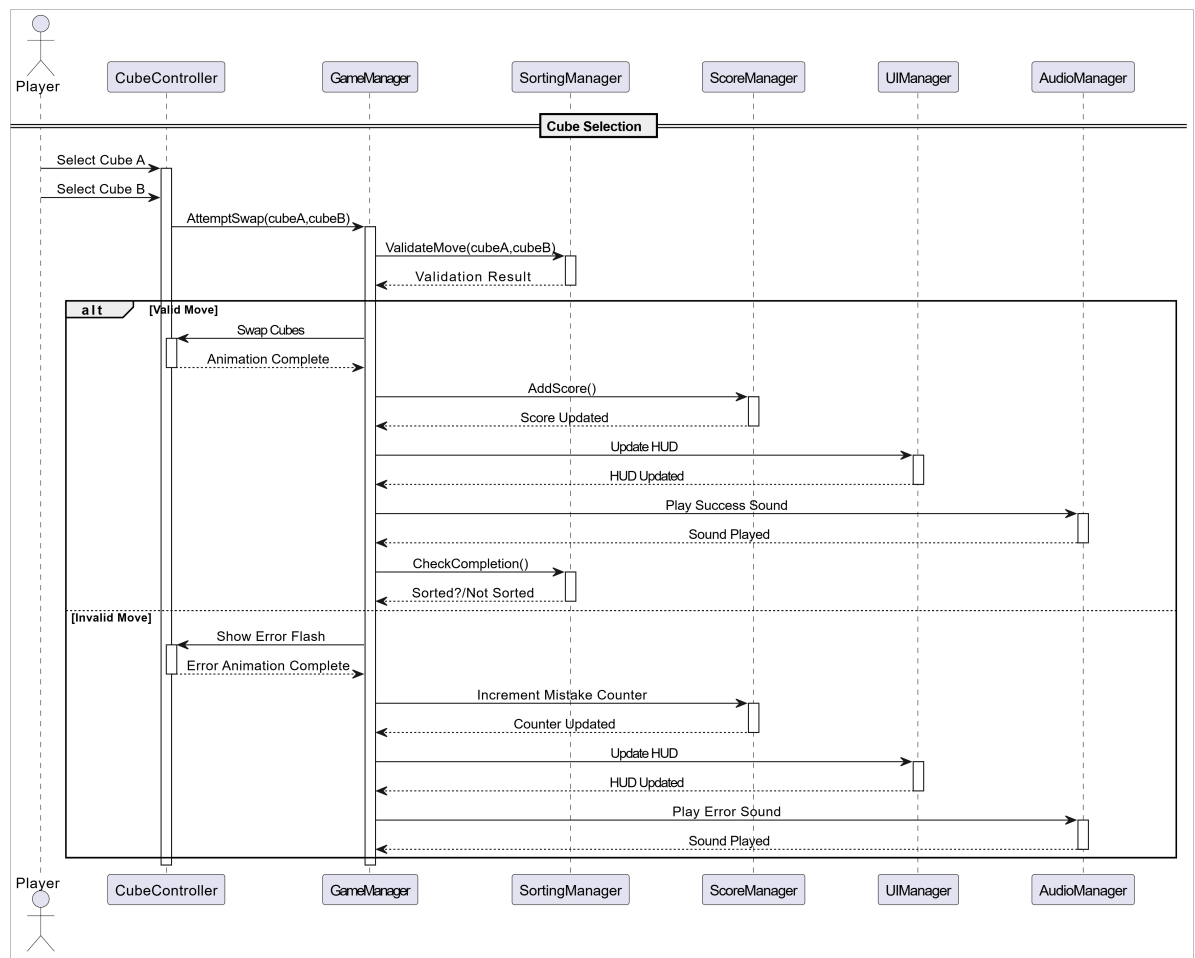


Figure 4: Sequence Diagram — Move Validation Process

Level Initialisation Sequence Diagram

Figure 5 illustrates the sequence of operations during level setup, showing how the GameManager coordinates cube spawning and the pre-computation of valid moves.

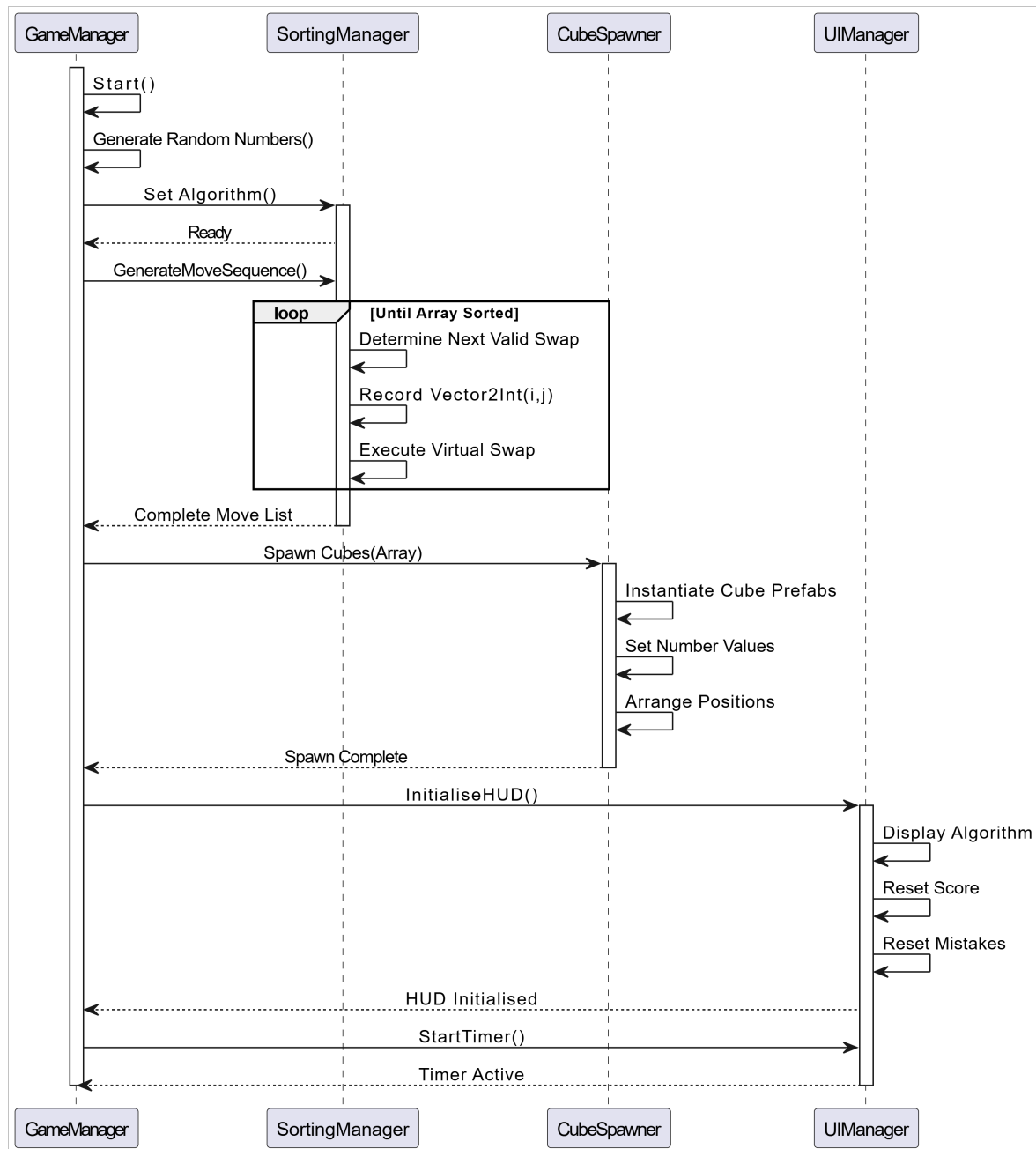


Figure 5: Sequence Diagram — Level Initialisation

Class Diagram

The class diagram in Figure 6 defines the structure of the five core C# classes. The **GameManager** acts as the central controller, holding references to all other managers. The **SortingManager** is a pure logic class responsible for generating and validating algorithm move sequences. The **ScoreManager** maintains the numeric state. The **LearningManager** populates the Learning Panel UI. The **CubeController** is attached to each individual cube prefab.

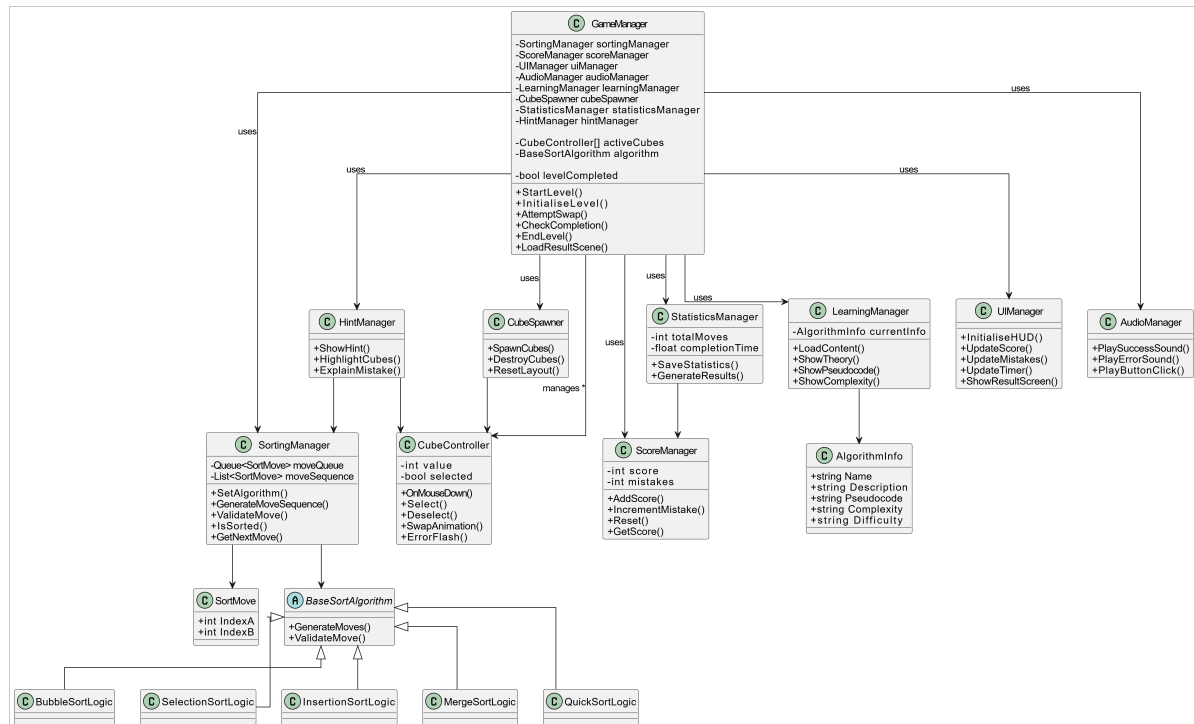


Figure 6: Class Diagram — Core Manager Architecture

Domain Model

The domain model in Figure 7 depicts the high-level conceptual relationships between the key entities in the system. A **Player** interacts with the system by playing one or more **Levels**. Each **Level** is governed by exactly one **Algorithm**. The algorithm defines a set of **Valid Moves** — the ordered sequence of swap operations that constitute a correct execution. Each level contains a collection of **Cubes**, each holding an integer value and position index.

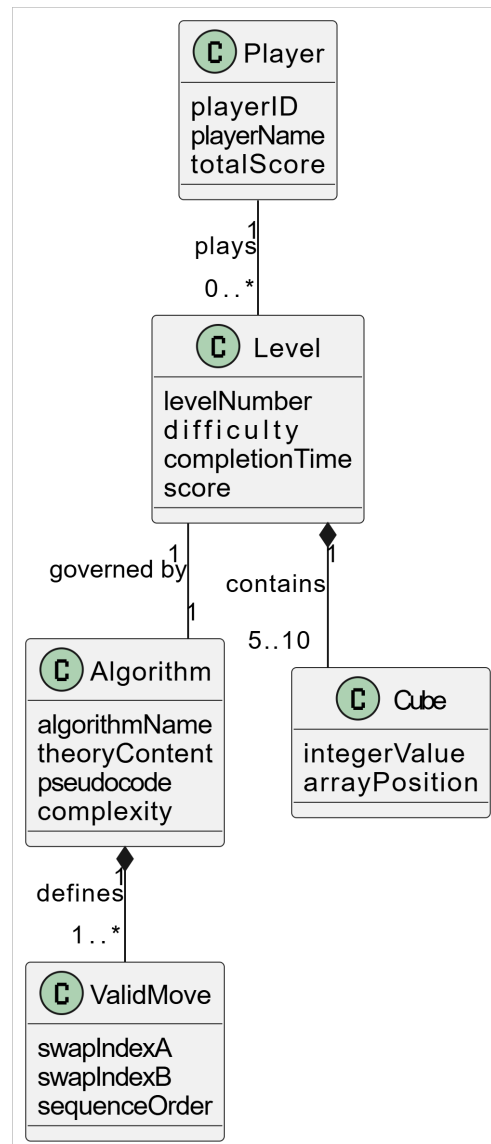


Figure 7: Domain Model

Entity Relationship Diagram

Database Design

Since the application operates as a standalone local game without a server, the term “database” refers to the persistent local storage managed by the **SaveSystem** module. Data is stored using Unity’s **PlayerPrefs** API or serialised JSON files. The ERD in Figure 8 models the two logical data entities that persist between sessions.

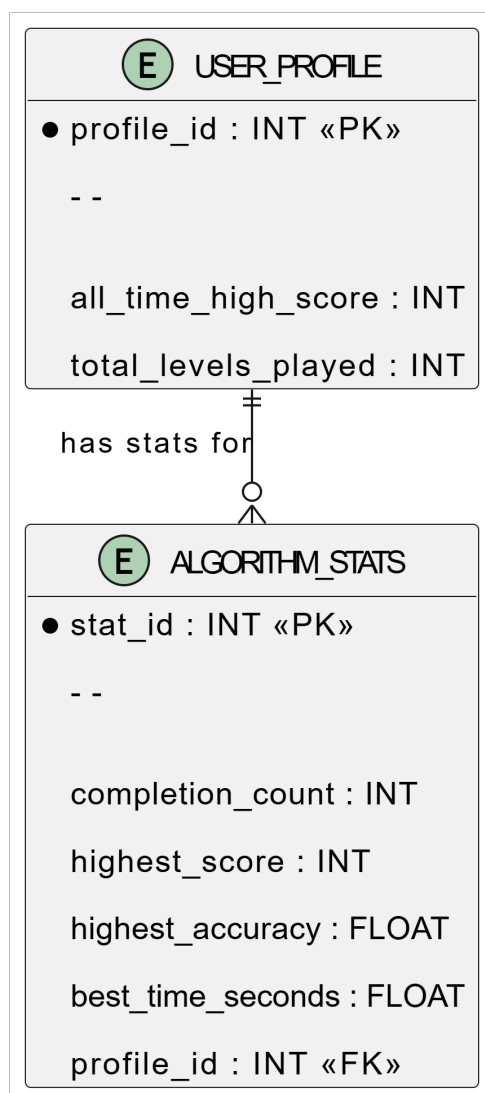


Figure 8: Entity Relationship Diagram — Local Storage

Database Tables

Table: USER_PROFILE

Table 27: USER_PROFILE Table

Attribute	Type	Constraint
profile_id	INT	Primary Key
player_name	VARCHAR(50)	NOT NULL
all_time_high_score	INT	DEFAULT 0
total_levels_played	INT	DEFAULT 0

Table: ALGORITHM_STATS

Table 28: ALGORITHM_STATS Table

Attribute	Type	Constraint
stat_id	INT	Primary Key
algorithm_name	VARCHAR(50)	NOT NULL
completion_count	INT	DEFAULT 0
highest_score	INT	DEFAULT 0
highest_accuracy	FLOAT	DEFAULT 0.0
best_time_seconds	FLOAT	NULLABLE
profile_id	INT	Foreign Key → USER_PROFILE

Package Diagram

Figure 9 illustrates the dependency structure between the script packages in the Unity project. The **UI Package** and **Gameplay Package** both depend on the **Managers Package**, which provides the Singleton controllers. The **Managers Package** delegates algorithmic computation to the **Algorithms Package** and data persistence to the **Data Package**.



Figure 9: Package Diagram

Architecture Diagram

Figure 10 presents the layered system architecture. The **Presentation Layer** handles all Unity rendering and direct user input capture; the **Logic Layer** contains the manager classes that process input, run validations, and maintain game state; and the **Data Layer** houses the algorithm computation classes and the local persistence system.

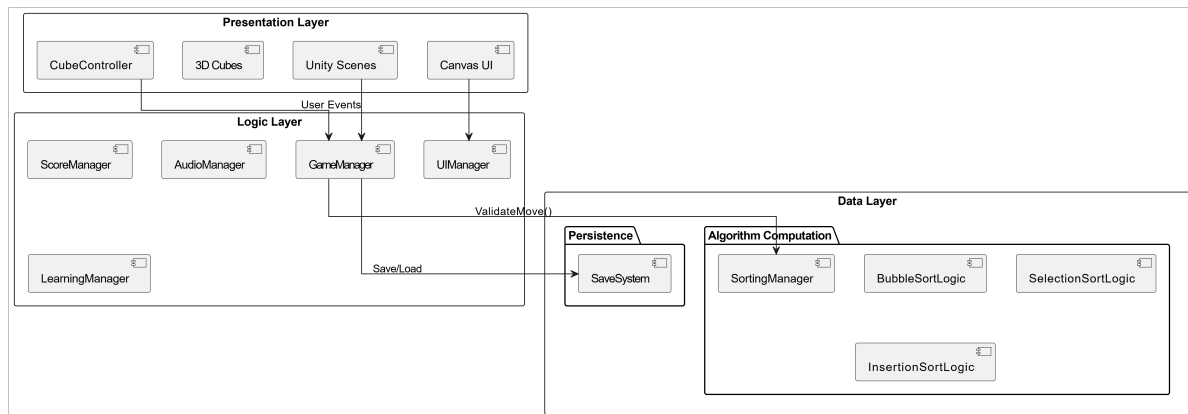


Figure 10: System Architecture Diagram

Deployment Diagram

Figure 11 illustrates the deployment of the application. The system is a self-contained client-side application that runs entirely on a single device with no server component. The Unity build contains the game engine runtime, all compiled C# logic via the Mono/IL2CPP backend, and the asset bundle. Persistent data is written to the local file system using PlayerPrefs.

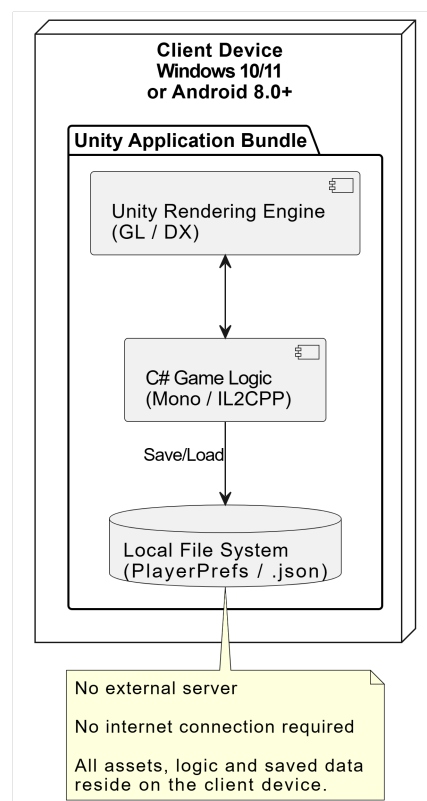


Figure 11: Deployment Diagram

Testing

Testing Introduction

Testing is the process of evaluating the system against its specified requirements to identify discrepancies between expected and actual behaviour. For the Game-Based Learning Platform, testing holds particular importance beyond typical software quality assurance. Because the platform’s purpose is educational, any error in the move validation logic does not merely constitute a software bug — it constitutes an incorrect educational experience that could actively mislead a student learning an algorithm. Consequently, the validation engine receives the most rigorous testing coverage.

Testing Strategy

The testing strategy for this project encompasses four levels of validation.

Unit Testing focuses on individual functions in isolation. The most critical unit tests verify that the `SortingManager`’s `GenerateMoves()` functions produce the exact correct swap sequence for known input arrays across all three supported algorithms.

Integration Testing verifies that the manager classes interact correctly with one another. Key integration tests confirm that a cube click correctly propagates from `CubeController` to `GameManager`, that `GameManager` correctly invokes `ScoreManager`, and that `AudioManager` receives events from `GameManager`.

System Testing involves complete end-to-end play-throughs of each algorithm level on target hardware, validating that the full gameplay flow from algorithm selection through to the result screen functions without errors.

User Acceptance Testing (UAT) involves a small group of computer science students from the target audience completing levels and providing structured feedback on usability, educational clarity, and perceived engagement.

Test Environment

Table 29: Test Environment

Component	Specification
Operating System	Windows 11
RAM	16 GB
GPU	DirectX 11 compatible GPU
Development Environment	Unity Editor 2022.3 LTS
Build Target (Primary)	Standalone Windows .exe
Build Target (Secondary)	Android APK (Android 8.0+)

Test Cases

Table 30: TC-01: Bubble Sort Valid Move

Field	Description
Test ID	TC-01
Module	Algorithm Validation — SortingManager
Preconditions	Game loaded in Bubble Sort mode. Array is [5, 3, 8, 1, 4]. SortingManager has pre-computed valid moves.
Test Steps	1. Click Cube at index 0 (value 5). 2. Click Cube at index 1 (value 3).
Expected Result	Cubes swap visually. Score increases by 10. Success sound plays.
Status	Passed

Table 31: TC-02: Bubble Sort Invalid Move

Field	Description
Test ID	TC-02
Module	Algorithm Validation — SortingManager
Preconditions	Same as TC-01. Correct first move is swap(0,1).
Test Steps	1. Click Cube at index 0 (value 5). 2. Click Cube at index 2 (value 8).
Expected Result	Cubes do NOT swap. Mistake counter increments by 1. Error sound plays. Red flash effect activates.
Status	Passed

Table 32: TC-03: Selection Sort Move Validation

Field	Description
Test ID	TC-03
Module	Algorithm Validation — SortingManager
Preconditions	Game loaded in Selection Sort mode. Array is [64, 25, 12, 22, 11]. Minimum is 11 at index 4. Correct first move is swap(0, 4).
Test Steps	1. Click Cube at index 0 (value 64). 2. Click Cube at index 4 (value 11).
Expected Result	Cubes swap. Score increments.
Status	Passed

Table 33: TC-04: Timer Timeout Handling

Field	Description
Test ID	TC-04
Module	Game Loop — GameManager
Preconditions	Gameplay Scene is active. Array not yet sorted.
Test Steps	Allow the countdown timer to reach zero without performing any swaps.
Expected Result	GameManager detects timeout. Player input is disabled. Result Scene loads with “Time’s Up” status. Statistics reflect 0 correct moves.
Status	Passed

Table 34: TC-05: Level Completion Trigger

Field	Description
Test ID	TC-05
Module	Game Loop — GameManager
Preconditions	Gameplay Scene active. Only one more valid swap remains to complete the sort.
Test Steps	Perform the final valid swap.
Expected Result	<code>GameManager.CheckLevelComplete()</code> returns true. Level-completion fanfare plays. <code>SaveSystem</code> writes updated statistics. Result Scene loads with final statistics displayed.
Status	Passed

Table 35: TC-06: Score and Accuracy Calculation

Field	Description
Test ID	TC-06
Module	ScoreManager
Preconditions	A level session has just ended with 10 correct moves and 2 incorrect moves.
Test Steps	Inspect the displayed accuracy value on the Result Screen.
Expected Result	Accuracy = $10/12 \times 100 = 83.33\%$. Score = $(10 \times 10) - (2 \times 5) = 90$. Grade = B.
Status	Passed

Table 36: TC-07: Data Persistence Across Sessions

Field	Description
Test ID	TC-07
Module	SaveSystem
Preconditions	Player has just completed a Bubble Sort level with a score of 120.
Test Steps	1. Note that high score of 120 is saved. 2. Close and relaunch the application. 3. Navigate to Main Menu or Statistics section.
Expected Result	Saved high score of 120 is correctly loaded and displayed after relaunch.
Status	Passed

Table 37: TC-08: Guided Practice Hint Display

Field	Description
Test ID	TC-08
Module	UI — Hint System
Preconditions	Game in Guided Practice mode with Bubble Sort. Array is [7, 2, 9, 1]. Next expected move is swap(0,1).
Test Steps	Observe the hint text and highlighted cubes without performing any interaction.
Expected Result	Hint text reads “Compare cube 1 and cube 2.” Cubes at index 0 and 1 display a visible highlight effect.
Status	Passed

Functional Testing

Functional testing was conducted by executing each functional requirement (FR-01 through FR-10) through its primary flow and at least one alternative or exception flow. All ten functional requirements passed their primary flow tests. The edge case of an already-sorted array being generated (addressed in FR-03 and NFR-07) was explicitly tested by temporarily forcing the level generator to produce a sorted array and confirming that the system detected and regenerated it.

Integration Testing

Integration testing confirmed the following interaction chains:

- `CubeController.OnMouseDown()` → `GameManager.SelectCube()` → `AttemptSwap()` → `SortingManager.ValidateMove()` — verified to function correctly with no null reference exceptions under rapid repeated tapping.
- `GameManager` → `AudioManager.PlaySound()` — verified that the correct audio clip is triggered for each event type (success, error, complete, click).
- `GameManager` → `ScoreManager.AddCorrectMove()` → `UIManager.UpdateScoreDisplay()` — verified that HUD values update within the same frame as the move validation response.

System Testing

Full end-to-end play-throughs were conducted for all three algorithms at both Easy and Medium difficulty settings. Each session was completed from the Main Menu through to the Result Screen. All algorithm validation sequences were verified by comparing observed gameplay behaviour against manually traced sorting sequences written on paper for the same input arrays.

Test Results

All eight documented test cases passed on the primary Windows build. The Android APK build was additionally tested on two Android devices and passed all test cases, with no degradation in frame rate or response time observed.

Limitations of Testing

- Load testing for concurrent users is not applicable, as this is a single-player local application.
- Testing was conducted on a limited set of two Android devices. Behaviour on devices with non-standard screen resolutions may differ from the tested configuration.
- User acceptance testing was conducted with a small sample of five students. A more statistically significant study would be needed to make formal claims about educational effectiveness.

Conclusion and Future Work

Conclusion

The Game-Based Learning Platform for Sorting Algorithms was successfully conceived, designed, and implemented in fulfilment of all ten specified functional requirements and five non-functional requirement categories. The platform translates the abstract computational logic of sorting algorithms into a tangible, interactive puzzle experience, requiring students to actively execute algorithmic decisions rather than passively observe them.

The most significant technical achievement of the project is the real-time move validation engine implemented within the **SortingManager**. By pre-computing the complete expected move sequence before gameplay begins, the system can immediately and accurately determine whether any player action conforms to the selected algorithm's rules. This mechanism is what transforms the application from a mere sorting game into a genuine educational tool: it guarantees that the system is always teaching correct algorithm behaviour, never allowing the student to reinforce incorrect strategies.

The architecture of the platform — built around a Singleton-based Manager pattern with clearly separated logic, presentation, and data layers — has proven robust, readable, and extensible throughout development. The codebase is well-positioned for future enhancement without requiring significant refactoring.

Future Enhancements

Integration of Advanced Algorithms: Merge Sort and Quick Sort are the most educationally significant algorithms not yet included. Their divide-and-conquer nature requires extending the cube interaction model to support array splitting and merging visualisations.

Multiplayer Competitive Mode: A split-screen or online competitive mode where two students race to sort the same array first would significantly increase engagement. Firebase Realtime Database would be a suitable backend for real-time synchronisation.

Global Leaderboards: Connecting the application to a cloud backend (e.g., Firebase) would enable a global leaderboard, adding a persistent competitive dimension that motivates repeated practice.

Adaptive Difficulty Engine: A machine-learning-based system that analyses the player's per-algorithm mistake patterns and automatically adjusts hint frequency, time limits, and array complexity would provide a personalised learning curve for each student.

Teacher Dashboard: A web-based dashboard where educators can view aggregate statistics from students using the platform would make it suitable for deployment in formal academic settings with assessment requirements.

Mobile Deployment: Publishing the application to the Google Play Store and Apple App Store would dramatically expand its accessible user base, making algorithm education available to anyone with a smartphone.

References

- [1] Bell, T., Witten, I. H., & Fellows, M. (1998). *Computer Science Unplugged: Off-line Activities and Games for All Ages*. Computer Science Unplugged.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [3] Unity Technologies. (2024). *Unity User Manual (2022.3 LTS)*. Retrieved from <https://docs.unity3d.com/Manual/index.html>
- [4] Demsar, J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7, 1–30.
- [5] Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From Game Design Elements to Gamefulness: Defining Gamification. *Proceedings of the 15th International Academic MindTrek Conference*, 9–15.
- [6] VisuAlgo. (2024). *Visualising Data Structures and Algorithms through Animation*. Retrieved from <https://visualgo.net>
- [7] HackerRank. (2024). *Practice Coding Problems*. Retrieved from <https://www.hackerrank.com>

Glossary of Key Terms

Algorithm: A finite, ordered sequence of well-defined instructions for solving a computational problem.

Bubble Sort: A comparison-based sorting algorithm that repeatedly steps through the array, compares adjacent elements, and swaps them if they are in the wrong order. Time complexity: $O(n^2)$ average and worst case.

Selection Sort: A sorting algorithm that divides the array into a sorted and an unsorted region, repeatedly selecting the smallest element from the unsorted region and moving it to the end of the sorted region. Time complexity: $O(n^2)$.

Insertion Sort: A sorting algorithm that builds a sorted array one element at a time by inserting each new element into its correct position within the already-sorted portion. Time complexity: $O(n^2)$ average, $O(n)$ best case.

Singleton Pattern: A software design pattern that restricts the instantiation of a class to a single object, providing a global access point to that instance.

DOTween: A fast and efficient tweening engine for Unity that enables smooth animations via code without requiring the Unity Animator component.

Gamification: The application of game-design elements and principles in non-game contexts to improve user engagement and motivation.

Source Code Snippet: Move Validation

The following C# method is the core of the educational accuracy guarantee described in FR-04. It compares the player's selected swap indices against the pre-computed `validMoves` list at the current sequence position.

Listing 1: SortingManager.cs — ValidateMove method

```
1 // SortingManager.cs
2 public bool ValidateMove(int indexA, int indexB)
3 {
4     if (currentMoveIndex >= validMoves.Count)
5         return false;
6
7     Vector2Int expectedMove = validMoves[currentMoveIndex];
8
9     bool isCorrect = (indexA == expectedMove.x && indexB == expectedMove.y)
10        || (indexA == expectedMove.y && indexB == expectedMove.x);
11
12     if (isCorrect)
13         currentMoveIndex++;
14
15     return isCorrect;
16 }
```

User Manual

Installation (Desktop)

Extract the provided .zip file to a folder on your computer. Double-click `GameBasedLearningPlatform.exe` to launch the application. No installation wizard or administrative privileges are required.

Installation (Android)

Enable installation from unknown sources in your device's security settings. Transfer the .apk file to your device and tap it to install.

How to Play

1. From the Main Menu, tap **Play Game**.
2. On the Algorithm Selection screen, tap the card for the algorithm you wish to study.
3. On the Learning Panel, read the description and watch the example animation. Tap **Play Now** when ready.
4. In the Game, tap two cubes to swap them. Sort the entire array before the timer runs out.
5. Restart the game to get a second chance. Apply what you have learned.
6. Review your score, accuracy, and grade on the Result Screen.